



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

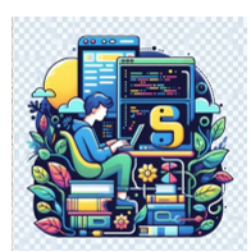
PROGRAMMING WITH PYTHON

Data Ingestion and Multiprocessing

Forking Process

Lekha A

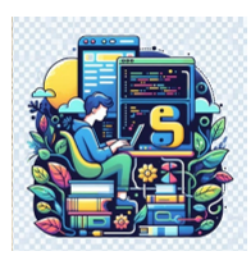
Computer Applications



PROGRAMMING WITH PYTHON

Process vs Threads

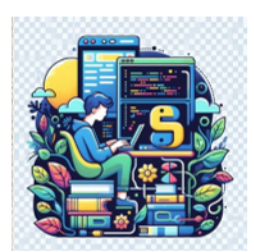
- A process is treated as a running program with its own PID, memory space, open files, and environment.
- A thread is run inside a process and memory is shared.
- A process is isolated and memory is not shared by default.



PROGRAMMING WITH PYTHON

Forking

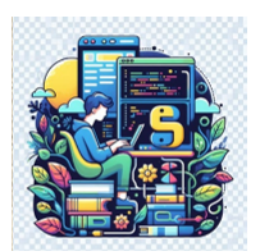
- Forking is a vital aspect of Unix and Linux environments.
- When a process performs a fork, it creates a replica of itself.
- More generally, in a multithreading setting, a fork refers to the duplication of a thread of execution, resulting in a child thread that originates from the parent thread.



PROGRAMMING WITH PYTHON

Forking

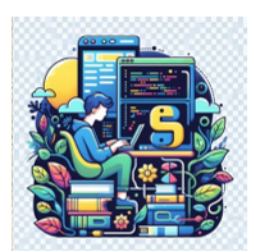
- They are similar but can be recognized as distinct.
- The fork operation generates a separate address space for the child process.
- The child process has an identical copy of all the memory from the parent process.
- The execution of the parent and child processes is independent of one another.



PROGRAMMING WITH PYTHON

Forking

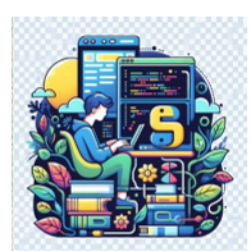
- The system function call `fork()` generates a duplicate of the process that invoked it.
- This duplicate operates as a child process of the original process.
- The child process inherits the data and code from the parent process.
- The child process is assigned its own process number (PID, Process Identifier) by the operating system.



PROGRAMMING WITH PYTHON

Forking

- It functions as an independent instance, meaning it operates separately from the parent process.
- The return value of `fork()` allows user to determine the process that is being run: a return value of 0 indicates that it is the child process, while a positive return value signifies that it is the parent process.
- A negative return value indicates that an error occurred during the forking attempt.



PROGRAMMING WITH PYTHON

Core idea of processes on Windows

```
PS D:\Courses\Programme\MCA\1 Sem\2025_26 Python\Course\Units\Unit 4\Programs\Forking> python process.py
Before process start
Parent process
Parent PID: 22432
Parent created child PID: 34312
Child process
Child PID: 34312
Child PPID: 22432
Parent finished after join
```

```
from multiprocessing import Process
'''It is used to create a separate OS level process.'''
import os
'''PID and PPID access is required.'''

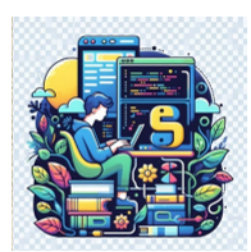
def child_task():
    '''This function will run in the child process only.'''
    print("Child process")
    print("Child PID:", os.getpid())
    print("Child PPID:", os.getppid())

if __name__ == "__main__":
    '''Execution entry point is protected.
    Required on Windows due to spawn model.
    Prevents infinite process creation.'''
    print("Before process start")

    p = Process(target=child_task)
    '''A Process object is created.
    child_task is registered as the child entry function.'''
    p.start()
    '''A new OS process is created.
    Python interpreter is started for the child.
    child_task begins execution in the child.'''

    print("Parent process")
    print("Parent PID:", os.getpid())
    print("Parent created child PID:", p.pid)

    p.join()
    '''The parent waits here.
    Execution is blocked until the child exits.'''
    print("Parent finished after join")
```



PROGRAMMING WITH PYTHON

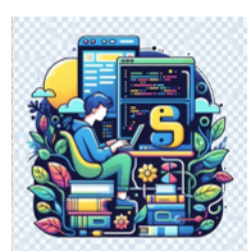
Controlling execution with Process - Windows

```
from multiprocessing import Process
'''Process is imported to create a new OS process.'''

import os
'''os is imported to read process IDs'''

import time
'''time is imported to pause execution.'''

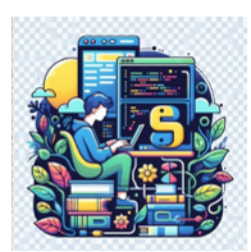
def child_task():
    '''child_task is defined.
    This function is executed inside the child process only.'''
    time.sleep(1)
    '''pauses the child for one second.'''
    '''delay is added so execution order is visible'''
    print("Child prints after 1 second")
    print("Child PID:", os.getpid())
    '''prints the child process ID.'''
    print("Child PPID:", os.getppid())
    '''prints the parent process ID'''
```



PROGRAMMING WITH PYTHON

Controlling execution with Process - Windows

```
if __name__ == "__main__":  
    '''The main guard is checked  
    Code inside runs only once in the parent.  
    This guard is required on Windows.  
    Infinite process spawning is prevented.'''  
  
    print("Parent prints immediately")  
    print("Parent PID:", os.getpid())  
    '''The parent PID is printed.  
    Only one process exists at this point.'''  
    p = Process(target=child_task)  
    '''A Process object is created.  
    child_task is registered as the child entry function.  
    No new process is started yet.'''  
  
    p.start()  
    '''A new OS process is created.  
    A new Python interpreter starts for the child.  
    child_task begins execution in parallel.'''  
  
    print("Parent created child PID:", p.pid)  
    print("Parent will wait now")  
  
    p.join()  
    '''The parent is blocked here.  
    The parent waits until the child finishes.'''  
    print("Parent prints last because join completed")
```



PROGRAMMING WITH PYTHON

Controlling execution with Process - Windows

```
PS D:\Courses\Programme\MCA\1 Sem\2025_26 Python\Course\Units\Unit 4\Programs\Forking> python process_control.py
```

```
Parent prints immediately
```

```
Parent PID: 19496
```

```
Parent created child PID: 30044
```

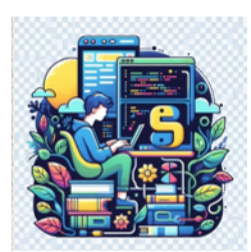
```
Parent will wait now
```

```
Child prints after 1 second
```

```
Child PID: 30044
```

```
Child PPID: 19496
```

```
Parent prints last because join completed
```

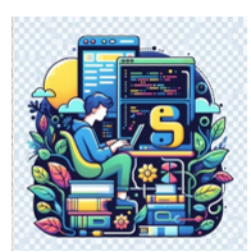


PROGRAMMING WITH PYTHON

Process Life Cycle - Windows

```
from multiprocessing import Process, freeze_support
'''Process is imported to create a new OS process'''
'''freeze_support is imported for Windows safety'''
import os
'''os is imported to read process identifiers'''
import time
'''time is imported to simulate work delay'''

def child_task():
    '''This function is executed inside the child process only.'''
    print("Child started", flush=True)
    print("Child PID:", os.getpid(), flush=True)
    print("Child PPID:", os.getppid(), flush=True)
    time.sleep(2)
    print("Child exiting", flush=True)
    '''flush=True forces immediate console output'''
```



PROGRAMMING WITH PYTHON

Process Life Cycle - Windows

```
def main():
    '''It avoids accidental execution during process spawning'''
    print("Parent started", flush=True)
    print("Parent PID:", os.getpid(), flush=True)
    '''At this point only one process exists'''

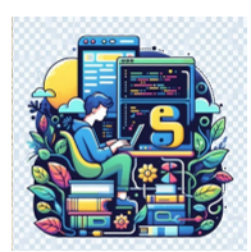
    p = Process(target=child_task)
    '''A Process object is created.
       child_task is registered as the target.'''
    p.start()
    '''Windows spawns a new Python interpreter.
       child_task begins execution in the child process.'''

    print("Child PID stored in parent:", p.pid, flush=True)
    print("Child alive after start:", p.is_alive(), flush=True)

    p.join()
    '''The parent is paused here.
       The parent waits until the child finishes execution.'''

    print("Child alive after join:", p.is_alive(), flush=True)
    print("Parent exiting safely", flush=True)

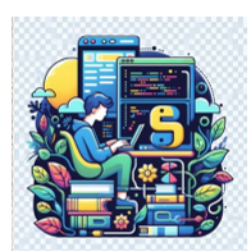
if __name__ == "__main__":
    freeze_support()
    '''freeze_support is executed for Windows compatibility.'''
    main()
    ''' main is called exactly once.
       Recursive process spawning is prevented.'''
```



PROGRAMMING WITH PYTHON

Process Life Cycle - Windows

```
PS D:\Courses\Programme\MCA\1 Sem\2025_26 Python\Course\Units\Unit 4\Programs\Forking> python process_lifecycle.py
Parent started
Parent PID: 32264
Child PID stored in parent: 23912
Child alive after start: True
Child started
Child PID: 23912
Child PPID: 32264
Child exiting
Child alive after join: False
Parent exiting safely
```

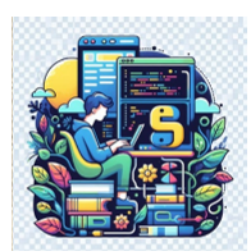


PROGRAMMING WITH PYTHON

Parallel Process - Windows

```
from multiprocessing import Process
'''Process is imported to create multiple OS level processes'''
import os
'''os is imported to read process IDs'''
def worker(n):
    '''This function runs inside a child process'''
    result = n * n
    '''A square is computed as n * n'''
    print("PID:", os.getpid(), "Task:", n, "Square:", result)
    ''' The child PID is printed using os.getpid
    Task value and computed result are printed'''

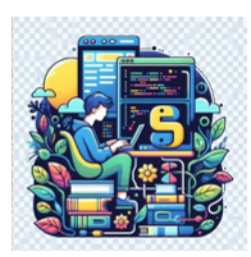
if __name__ == "__main__":
    tasks = [2, 3, 4, 5, 6]
    '''A list of independent tasks is created'''
    processes = []
    '''A list named processes is initialized to track children'''
    for n in tasks:
        '''A Process object is created for each task'''
        p = Process(target=worker, args=(n,))
        '''worker is assigned as the target function'''
        p.start()
        '''A new OS process is spawned for each task'''
        processes.append(p)
    for p in processes:
        '''All child processes run independently'''
        p.join()
        '''The parent waits until all children finish.'''
    print("Parent PID:", os.getpid(), "All tasks finished")
```



PROGRAMMING WITH PYTHON

Parallel Process - Windows

```
PS D:\Courses\Programme\MCA\1 Sem\2025_26 Python\Course\Units\Unit 4\Programs\Forking> python process_parallel.py
PID: 18912 Task: 2 Square: 4
PID: 24428 Task: 3 Square: 9
PID: 37380 Task: 4 Square: 16
PID: 28016 Task: 5 Square: 25
PID: 14208 Task: 6 Square: 36
Parent PID: 36804 All tasks finished
```



PROGRAMMING WITH PYTHON

Use case: log analysis with merging

- A log file is processed in parallel.
- Each process counts log levels in its chunk.
- Results are merged in the parent.



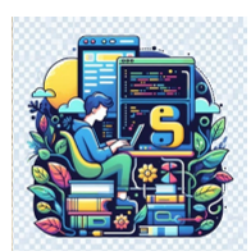
PROGRAMMING WITH PYTHON

Use case: log analysis with merging

```
from multiprocessing import Process
import os

LOG_FILE = "access.log"
NUM_PROCS = 3
LEVELS = ["INFO", "WARN", "ERROR"]
'''LOG_FILE stores the input file name.
NUM_PROCS sets how many processes are created.
LEVELS defines valid log categories.'''

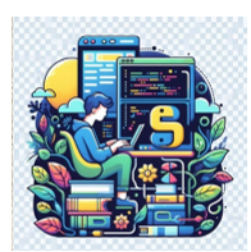
def split_lines(lines, k):
    '''Input lines are divided into k chunks.
    Chunk size is calculated using ceiling division.
    Each chunk holds a contiguous block of lines.
    Balanced workload distribution is achieved.'''
    size = (len(lines) + k - 1) // k
    return [lines[i*size:(i+1)*size] for i in range(k)]
```



PROGRAMMING WITH PYTHON

Use case: log analysis with merging

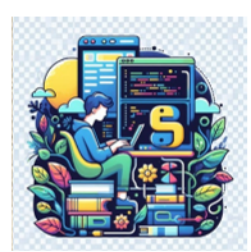
```
def worker(chunk, out_file):  
    '''This function runs in a child process'''  
    counts = {lvl: 0 for lvl in LEVELS}  
    '''A dictionary is created for local counts'''  
    for line in chunk:  
        parts = line.strip().split()  
        '''Each line is split into tokens'''  
        if not parts:  
            continue  
        level = parts[0]  
        '''The first token is treated as log level'''  
        if level in counts:  
            '''Counts are updated only for known levels'''  
            counts[level] += 1  
  
    with open(out_file, "w") as f:  
        '''Results are written to a temporary file'''  
        for lvl in LEVELS:  
            f.write(f"{lvl} {counts[lvl]}\n")
```



PROGRAMMING WITH PYTHON

Use case: log analysis with merging

```
if __name__ == "__main__":  
    with open(LOG_FILE, "r") as f:  
        '''The entire log file is read into memory'''  
        lines = f.readlines()  
        chunks = split_lines(lines, NUM_PROCS)  
        '''Lines are passed to the splitter.'''  
        '''Lines are split into NUM_PROCS chunks.'''  
        '''Each chunk is independent'''  
  
        processes = []  
        temp_files = []  
  
        for i, chunk in enumerate(chunks):  
            '''One temporary output file is assigned per chunk'''  
            out_file = f"part_{i}.txt"  
            temp_files.append(out_file)  
  
            p = Process(target=worker, args=(chunk, out_file))  
            '''One Process is created per chunk'''  
            '''worker is passed the chunk and file name'''  
            p.start()  
            ''' p.start launches each child immediately'''  
            processes.append(p)  
            '''Process objects are stored for tracking'''
```



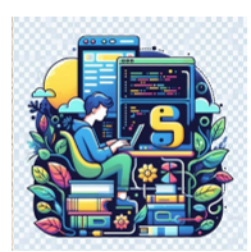
PROGRAMMING WITH PYTHON

Use case: log analysis with merging

```
for p in processes:
    '''All workers run concurrently.
    Each worker counts its own chunk only.
    Output order is irrelevant.'''
    p.join()
    '''The parent waits for every child.
    No merging starts until all workers finish.
    Clean synchronization is enforced'''

final_counts = {lvl: 0 for lvl in LEVELS}
'''A final_counts dictionary is initialized'''
for tf in temp_files:
    with open(tf, "r") as f:
        '''Each temporary file is read sequentially'''
        for line in f:
            lvl, val = line.strip().split()
            final_counts[lvl] += int(val)
            '''Partial counts are added to final totals'''
    os.remove(tf)
    '''Temporary files are deleted after reading'''

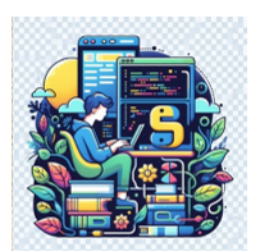
print("Merged counts:", final_counts)
```



PROGRAMMING WITH PYTHON

Use case: log analysis with merging

```
PS D:\Courses\Programme\MCA\1 Sem\2025_26 Python\Course\Units\Unit 4\Programs\Forking> python process_log.py  
Merged counts: {'INFO': 12, 'WARN': 4, 'ERROR': 4}
```



PROGRAMMING WITH PYTHON

Recap

- Process vs Threads
- Forking
- Processes
- Controlling Execution with Process
- Process Life Cycle
- Parallel Process
- Use case: log analysis with merging
- Windows or Linux/Unix/Mac OS



THANK YOU

Lekha A
Computer Applications